

BEYOND THE TEXTBOOK · GUIDE 03

Class Guestbook with Supabase

Save data in a real cloud database and show it on your website.

TIME NEEDED 45–60 minutes	LEVEL Beginner to intermediate
YOU NEED A GitHub or email account and the starter files	STARTER FILES examples/03-supabase-guestbook/

WHAT YOU WILL BUILD

A guestbook page. Visitors type their name and a message, click **Post**, and the message is saved in a Supabase database. Every visitor then sees the full list of messages, newest first.

Key ideas in plain words

Word	Meaning	In our guestbook
Database	An organised place to store data online	Our Supabase project
Table	Like one sheet in Excel or Google Sheets	<code>feedback</code>
Column	One type of information	<code>id, name, message, created_at</code>
Row	One record	One visitor's message
API	The door your web page uses to talk to the database	<code>supabase-js</code> handles it for us
Publishable key	A public key that identifies your project; safe in a web page	Pasted into <code>app.js</code>
Row Level Security (RLS)	Rules that decide who may read or write each row	Anyone may read and add; only a signed-in admin may delete (Step 10)

GOOD TO KNOW

Formspree vs Supabase: Formspree **emails** you a message. Supabase **stores** it so your page can show it back to everyone.

GOOD TO KNOW

Free plan (checked September 2026): 2 active projects, 500 MB database, 1 GB file storage. **Free projects pause after 1 week without activity.** Details: supabase.com/pricing.

Step 1 Create a Supabase account

1. Go to supabase.com and click **Start your project**.
2. Sign up with **GitHub** (recommended, since you will use GitHub in Guide 04) or with your email.
3. Confirm your email if asked.

Step 2 Create a new project

1. In the dashboard, click **New project**. Choose your organisation (Supabase creates one for you).
2. **Project name:** `class-guestbook`.
3. **Database password:** click **Generate a password** and save it somewhere safe. You will not need it for this guide, but you cannot see it again later.
4. **Region:** choose the one closest to your users, for example **South Asia (Mumbai)**.
5. Keep the **Free** plan and click **Create new project**. Setup takes one to two minutes.

Step 3 Create the table with SQL

Instead of clicking through many settings, we run one ready-made script, `setup.sql`. It creates the table, switches on security, adds the rules, and inserts two sample messages.

1. In the left sidebar, open **SQL Editor**.
2. Click **New query** (or the **+** button).
3. Open `setup.sql` from the starter folder, copy **everything**, and paste it into the editor.
4. Click **Run** (or press `Ctrl + Enter`). You should see **Success. No rows returned**.

Part of setup.sql	What it does
<code>create table ... feedback</code>	Makes the table with four columns. <code>id</code> and <code>created_at</code> fill themselves in automatically.
<code>check (char_length(...))</code>	Rejects empty names and messages longer than 300 characters.
<code>enable row level security</code>	Locks the table. Nothing is allowed until a policy says so.
<code>create policy ... for select</code>	Rule 1: anyone visiting the site may read messages.
<code>create policy ... for insert</code>	Rule 2: anyone may add a message. There is still no rule for editing, and only a signed-in admin may delete (see the delete policy below).
<code>grant select, insert ... to anon</code>	Lets the website use the table through the API. Some newer Supabase projects do not grant this automatically, so running this line yourself is always a safe step.
<code>create policy ... for delete</code>	Rule 3: only a signed-in (authenticated) user may delete a message — this powers the admin panel (Step 10).
<code>insert into ... values</code>	Adds two sample messages so the page is not empty.

TIP

If a “destructive operation” or “potential issue” warning appears, read it and confirm. The script is safe to run more than once.

Step 4 Check your table

Open **Table Editor** in the left sidebar and click the `feedback` table. You should see the two sample rows from Asha and Rohan, and a label showing that **RLS is enabled**.

Step 5 Copy your project URL and publishable key

1. Click the **Connect** button at the top of the dashboard, or go to **Project Settings** → **API Keys**.
2. Copy the **Project URL**. It looks like `https://abcdefghijklmnopqrstuvwxyz.supabase.co`.
3. Copy the **publishable key**. It starts with `sb_publishable_`.

GOOD TO KNOW

Older projects may show a **legacy anon key** (a long text starting with `eyJ`). It works the same way in this example.

BE CAREFUL

Never use the **secret key** (`sb_secret_...`) or the **service_role** key in a web page or on GitHub. Anyone who has it can read, change and delete **all** your data.

Step 6 Paste them into app.js

Open `examples/03-supabase-guestbook/app.js` and replace the two placeholder values at the top:

app.js — lines 10 and 11

```
// Before
const SUPABASE_URL = "https://YOUR-PROJECT-ID.supabase.co";
const SUPABASE_KEY = "YOUR_PUBLISHABLE_KEY";

// After (example)
const SUPABASE_URL = "https://abcdefghijklmnopqrstuvwxyz.supabase.co";
const SUPABASE_KEY = "sb_publishable_AbC123...";
```

Keep the quotation marks. Save the file.

Step 7 Run and test the guestbook

1. Right-click `index.html` in VS Code and choose **Open with Live Server**.
2. You should see the two sample messages under **Messages (2)**.

3. Type your name and a message, then click **Post message**.
4. A green “Thanks, ...! Your message is posted.” appears and your message moves to the top of the list.
5. Go back to **Table Editor** in Supabase. Your new row is there.

TIP

Press **F12** and open the **Console** tab. If anything goes wrong, the full error message appears there.

Step 8 Understand the code

1. Load the library and connect

index.html and app.js

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>

const db = supabase.createClient(SUPABASE_URL, SUPABASE_KEY);
```

The first line downloads Supabase's JavaScript library. `createClient` connects to your project and gives us an object called `db`.

2. Read messages

app.js — loadMessages()

```
const { data, error } = await db
  .from("feedback")                // which table
  .select("*")                     // all columns
  .order("created_at", { ascending: false }) // newest first
  .limit(50);                      // at most 50 rows
```

This is like saying: “From the feedback table, give me everything, newest first, maximum 50.” The rows come back in `data`; if something fails, the reason is in `error`.

3. Save a message

app.js — form submit

```
const { error } = await db
  .from("feedback")
  .insert({ name: name, message: message });
```

`insert` adds one new row. We only send `name` and `message`; the database fills in `id` and `created_at`.

4. Show messages safely

```
app.js — showMessages()
who.textContent = row.name;
text.textContent = row.message;
```

We use `textContent`, not `innerHTML`. If someone types HTML or a script into the message box, it is shown as plain text instead of being run on other visitors' screens.

GOOD TO KNOW

`async` and `await` mean “wait for the database to answer before moving to the next line”. Talking to a server takes a moment, and `await` handles that waiting for you.

Step 9 Publish your guestbook

Upload the folder to GitHub Pages with **Guide 04**. The publishable key in `app.js` is safe to publish because RLS only allows reading and adding messages.

Step 10 Add an admin panel (optional)

The guestbook already lets anyone read and add messages. `admin.html` is a separate page where a signed-in teacher can review every message and delete anything inappropriate.

In your Supabase dashboard, open Authentication → Users → Add user, and create an email and password for yourself. This is the only account that can sign in on `admin.html` — nobody else can delete a row, even if they find the page's address.

`setup.sql` already includes the rule this needs: a delete policy scoped to authenticated (signed-in) users only, with a matching grant. If your project was created before this rule existed, simply run `setup.sql` again — it is safe to run more than once.

Open `examples/03-supabase-guestbook/admin.html` with Live Server, sign in with the account you just created, and you will see every message with a Delete button next to it.

`admin.js` needs the same `SUPABASE_URL` and `SUPABASE_KEY` values you pasted into `app.js` in Step 6. Paste them there too.

Step 11 Keep your free project awake

Free Supabase projects **pause after one week without activity**. If your guestbook stops loading, open the Supabase dashboard and click **Restore project** (or **Resume**). Your data is kept.

Challenges: take it further

- Add a `class_name` column (for example “12-A”) and a matching drop-down in the form. Remember to update `setup.sql`, the `insert` and the display code.
- Show a friendly time such as “5 minutes ago” instead of the full date.
- Add a character counter that shows how many of the 300 characters are left.

- Go further with Supabase Auth: the admin panel already uses it for deleting messages — try requiring visitors to sign in before they can post too: supabase.com/docs/guides/auth.

ASK AN AI HELPER (Claude, Gemini, Copilot or Antigravity)

“Explain this Supabase setup.sql line by line for a beginner.”

“I get 'new row violates row-level security policy'. Here is my SQL. What is missing?”

“Add a class_name column to my Supabase guestbook: give me the SQL and the changes to app.js.”

“Is it safe to put my Supabase publishable key in a public GitHub repository? Explain RLS.”

Troubleshooting

Error or problem	Likely cause and fix
“Setup needed: paste your Supabase URL...”	The placeholder values are still in <code>app.js</code> (Step 6).
<code>permission denied for table feedback</code>	The <code>grant</code> line did not run. Run <code>grant select, insert on public.feedback to anon;</code> in the SQL Editor.
<code>new row violates row-level security policy</code>	The insert policy is missing. Run <code>setup.sql</code> again.
<code>relation "public.feedback" does not exist</code>	The table was not created, or has a different name. Run <code>setup.sql</code> and check the spelling in <code>app.js</code> .
<code>Invalid API key</code>	The key was copied incompletely or it is the wrong key. Copy the publishable key again.
<code>supabase is not defined</code>	The library did not load. Check the <code><script src="https://cdn.jsdelivr.net/..."></code> line and your internet connection.
Messages stopped loading after some days	The free project is paused. Restore it from the dashboard (Step 11).
Can't sign in on admin.html	No admin account exists yet, or the email/password is wrong. Create one in Supabase: Authentication → Users → Add user (Step 10).

Final checklist

- ☐ My Supabase project is created and `setup.sql` ran successfully.
- ☐ RLS is enabled on the `feedback` table.
- ☐ `app.js` contains my Project URL and my **publishable** key, not the secret key.
- ☐ I can post a message and see it in both the page and the Table Editor.
- ☐ The guestbook works on my published GitHub Pages site.

- ☐ (Optional) I created an admin account and can sign in to `admin.html` to delete a message.

Appendix: complete code

setup.sql

```
-- =====
-- Class Guestbook: database setup for Supabase
-- HOW TO USE: Supabase dashboard -> SQL Editor -> New query
--             Paste this whole file -> click "Run"
-- =====

-- 1. Create the table that stores guestbook messages
create table if not exists public.feedback (
  id          bigint generated always as identity primary key,
  name        text not null check (char_length(name) between 1 and 50),
  message     text not null check (char_length(message) between 1 and 300),
  created_at  timestampz not null default now()
);

-- 2. Turn ON Row Level Security (RLS).
--    With RLS on, nobody can read or write the table
--    unless a policy below allows it.
alter table public.feedback enable row level security;

-- 3. Policy: anyone visiting the website may READ messages
--    ("drop ... if exists" lets you run this file again without errors)
drop policy if exists "Anyone can read feedback" on public.feedback;
create policy "Anyone can read feedback"
  on public.feedback
  for select
  to anon
  using (true);

-- 4. Policy: anyone visiting the website may ADD a message
--    (they cannot edit or delete, because we add no policy for that)
drop policy if exists "Anyone can add feedback" on public.feedback;
create policy "Anyone can add feedback"
  on public.feedback
  for insert
  to anon
  with check (true);

-- 5. Allow the website (the "anon" role) to use the table through the API.
--    Some newer Supabase projects do not do this automatically
--    for new tables, so we do it ourselves. Running it earlier is harmless.
grant select, insert on public.feedback to anon;

-- 6. Policy: only a SIGNED-IN user may DELETE a message.
--    This is what powers admin.html. Create the admin account in
--    Supabase: Authentication -> Users -> Add user (email + password).
--    Nobody can delete until they sign in with that account.
drop policy if exists "Signed-in users can delete feedback" on public.feedback;
create policy "Signed-in users can delete feedback"
  on public.feedback
  for delete
  to authenticated
  using (true);

grant select, delete on public.feedback to authenticated;

-- 7. Optional: add two sample messages so the page is not empty
insert into public.feedback (name, message) values
  ('Asha', 'Loved the quiz page!'),
  ('Rohan', 'Add a dark mode please');
```

index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Class Guestbook | Supabase Example</title>
  <meta name="description" content="A class guestbook that saves messages in a Supabase
database and shows them to every visitor.">
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <main class="container">
    <p class="kicker">Supabase example</p>
    <h1>Class Guestbook</h1>
    <p class="intro">Leave a message for the class. It is saved in a Supabase database and
everyone can see it.</p>

    <!-- 1. Form to ADD a message -->
    <form id="guestbook-form" class="panel">
      <label for="name">Your name</label>
      <input type="text" id="name" maxlength="50" placeholder="e.g. Meera" required>

      <label for="message">Your message</label>
      <textarea id="message" rows="3" maxlength="300" placeholder="Say something nice..."
required></textarea>

      <button type="submit" id="post-btn">Post message</button>
      <p id="status" role="status"></p>
    </form>

    <!-- 2. List where saved messages are SHOWN -->
    <section>
      <h2>Messages <span id="count"></span></h2>
      <ul id="message-list">
        <li class="empty">Loading messages...</li>
      </ul>
    </section>
  </main>

  <!-- 3. Load the Supabase library from a CDN (no installation needed) -->
  <script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>

  <!-- 4. Our own code -->
  <script src="app.js"></script>
</body>
</html>

```

app.js

```

// =====
// Class Guestbook with Supabase
// =====

// STEP 1: Paste your own project details here.
// Find them in Supabase: Project Settings -> API Keys (or the "Connect" button).
// The PUBLISHABLE key (starts with sb_publishable_) is safe in a web page
// because Row Level Security protects the table.
// NEVER paste the SECRET key (sb_secret_...) or the service_role key here.
const SUPABASE_URL = "https://YOUR-PROJECT-ID.supabase.co";
const SUPABASE_KEY = "YOUR_PUBLISHABLE_KEY";

// STEP 2: Connect to Supabase
const db = supabase.createClient(SUPABASE_URL, SUPABASE_KEY);

```

```

// STEP 3: Find the elements on the page
const form = document.getElementById("guestbook-form");
const nameInput = document.getElementById("name");
const messageInput = document.getElementById("message");
const button = document.getElementById("post-btn");
const statusText = document.getElementById("status");
const list = document.getElementById("message-list");
const count = document.getElementById("count");

// -----
// READ: get all messages from the "feedback" table
// -----
async function loadMessages() {
  const { data, error } = await db
    .from("feedback")
    .select("*")
    .order("created_at", { ascending: false }) // newest first
    .limit(50);

  if (error) {
    list.innerHTML = "";
    addEmptyLine("Could not load messages: " + error.message);
    console.error(error);
    return;
  }

  showMessages(data);
}

// -----
// WRITE: save a new message when the form is submitted
// -----
form.addEventListener("submit", async function (event) {
  event.preventDefault(); // stop the page from reloading

  const name = nameInput.value.trim();
  const message = messageInput.value.trim();
  if (!name || !message) return;

  button.disabled = true;
  button.textContent = "Posting...";

  const { error } = await db
    .from("feedback")
    .insert({ name: name, message: message });

  if (error) {
    setStatus("Could not save: " + error.message, "error");
    console.error(error);
  } else {
    setStatus("Thanks, " + name + "! Your message is posted.", "success");
    form.reset();
    loadMessages(); // refresh the list so the new message appears
  }

  button.disabled = false;
  button.textContent = "Post message";
});

// -----
// Helpers that build the list on the page
// -----
function showMessages(rows) {
  list.innerHTML = "";
  count.textContent = "(" + rows.length + ")";

  if (rows.length === 0) {

```

```

    addEmptyLine("No messages yet. Be the first!");
    return;
  }

  rows.forEach(function (row) {
    const item = document.createElement("li");

    const who = document.createElement("strong");
    who.textContent = row.name; //.textContent keeps visitors from injecting HTML

    const when = document.createElement("span");
    when.className = "time";
    when.textContent = new Date(row.created_at).toLocaleString("en-IN");

    const text = document.createElement("p");
    text.textContent = row.message;

    item.append(who, when, text);
    list.appendChild(item);
  });
}

function addEmptyLine(text) {
  const item = document.createElement("li");
  item.className = "empty";
  item.textContent = text;
  list.appendChild(item);
}

function setStatus(message, type) {
  statusText.textContent = message;
  statusText.className = type;
}

// STEP 4: Load messages as soon as the page opens
if (SUPABASE_URL.includes("YOUR-PROJECT-ID")) {
  list.innerHTML = "";
  addEmptyLine("Setup needed: paste your Supabase URL and publishable key into app.js.");
  button.disabled = true;
} else {
  loadMessages();
}

```

admin.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Admin | Class Guestbook</title>
  <meta name="description" content="Admin panel to review and remove messages in the
Supabase class guestbook.">
  <meta name="robots" content="noindex, nofollow">
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <main class="container">
    <p class="kicker">Supabase example &middot; Admin</p>
    <h1>Guestbook Admin</h1>
    <p class="intro">Sign in with the admin account to review messages and remove anything
inappropriate.</p>

    <!--
    HOW THIS WORKS
    - This page is NOT secret. Anyone who finds the URL can open it.
    - What keeps it safe is Supabase Auth + Row Level Security (RLS):

```

```

    only someone who signs in with a real admin email/password
    (created by the teacher in the Supabase dashboard) can delete rows.
    Everyone else can only read and post, same as before.
-->

<!-- 1. Sign-in form (shown when nobody is logged in) -->
<form id="login-form" class="panel">
  <label for="email">Admin email</label>
  <input type="email" id="email" placeholder="teacher@example.com" required>

  <label for="password">Password</label>
  <input type="password" id="password" required>

  <button type="submit" id="login-btn">Sign in</button>
  <p id="login-status" role="status"></p>
</form>

<!-- 2. Admin panel (shown after sign-in) -->
<section id="admin-panel" class="panel hidden">
  <div class="admin-bar">
    <p id="who">Signed in</p>
    <button type="button" id="logout-btn">Sign out</button>
  </div>
  <h2>Messages <span id="count"></span></h2>
  <ul id="message-list">
    <li class="empty">Loading messages...</li>
  </ul>
</section>
</main>

<!-- Load the Supabase library from a CDN (no installation needed) -->
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>

<!-- Our own code -->
<script src="admin.js"></script>
</body>
</html>

```

admin.js

```

// =====
// Class Guestbook: Admin panel
// Lets a signed-in teacher/admin review messages and delete
// anything inappropriate. Everyone else only gets read+post
// access (see app.js and setup.sql).
// =====

// STEP 1: Same project details as app.js. Paste your own here too.
const SUPABASE_URL = "https://YOUR-PROJECT-ID.supabase.co";
const SUPABASE_KEY = "YOUR_PUBLISHABLE_KEY";

const db = supabase.createClient(SUPABASE_URL, SUPABASE_KEY);

// STEP 2: Find the elements on the page
const loginForm = document.getElementById("login-form");
const emailInput = document.getElementById("email");
const passwordInput = document.getElementById("password");
const loginBtn = document.getElementById("login-btn");
const loginStatus = document.getElementById("login-status");

const adminPanel = document.getElementById("admin-panel");
const whoText = document.getElementById("who");
const logoutBtn = document.getElementById("logout-btn");
const list = document.getElementById("message-list");
const count = document.getElementById("count");

// -----
// SIGN IN: only an account created by the teacher in the

```

```

// Supabase dashboard (Authentication -> Users -> Add user) works.
// -----
loginForm.addEventListener("submit", async function (event) {
  event.preventDefault();

  if (SUPABASE_URL.includes("YOUR-PROJECT-ID")) {
    setLoginStatus("Setup needed: paste your Supabase URL and publishable key into
admin.js.", "error");
    return;
  }

  loginBtn.disabled = true;
  loginBtn.textContent = "Signing in...";

  const { error } = await db.auth.signInWithPassword({
    email: emailInput.value.trim(),
    password: passwordInput.value
  });

  if (error) {
    setLoginStatus("Could not sign in: " + error.message, "error");
  }

  loginBtn.disabled = false;
  loginBtn.textContent = "Sign in";
});

logoutBtn.addEventListener("click", async function () {
  await db.auth.signOut();
});

// -----
// Show the login form or the admin panel depending on
// whether someone is currently signed in.
// -----
db.auth.onAuthStateChange(function (_event, session) {
  if (session) {
    loginForm.classList.add("hidden");
    adminPanel.classList.remove("hidden");
    whoText.textContent = "Signed in as " + session.user.email;
    loadMessages();
  } else {
    loginForm.classList.remove("hidden");
    adminPanel.classList.add("hidden");
    loginForm.reset();
  }
});

// -----
// READ: get every message, newest first
// -----
async function loadMessages() {
  const { data, error } = await db
    .from("feedback")
    .select("*")
    .order("created_at", { ascending: false });

  if (error) {
    list.innerHTML = "";
    addEmptyLine("Could not load messages: " + error.message);
    console.error(error);
    return;
  }

  showMessages(data);
}

// -----

```

```

// DELETE: remove one row. RLS only allows this for a
// signed-in (authenticated) user - see setup.sql.
// -----
async function deleteMessage(id) {
  const { error } = await db.from("feedback").delete().eq("id", id);

  if (error) {
    alert("Could not delete: " + error.message);
    console.error(error);
    return;
  }

  loadMessages();
}

// -----
// Helpers that build the list on the page
// -----
function showMessages(rows) {
  list.innerHTML = "";
  count.textContent = "(" + rows.length + ")";

  if (rows.length === 0) {
    addEmptyLine("No messages yet.");
    return;
  }

  rows.forEach(function (row) {
    const item = document.createElement("li");

    const who = document.createElement("strong");
    who.textContent = row.name; // textContent keeps visitors from injecting HTML

    const when = document.createElement("span");
    when.className = "time";
    when.textContent = new Date(row.created_at).toLocaleString("en-IN");

    const text = document.createElement("p");
    text.textContent = row.message;

    const removeBtn = document.createElement("button");
    removeBtn.type = "button";
    removeBtn.className = "delete-btn";
    removeBtn.textContent = "Delete";
    removeBtn.addEventListener("click", function () {
      if (confirm("Delete this message from " + row.name + "?")) {
        deleteMessage(row.id);
      }
    });

    item.append(who, when, text, removeBtn);
    list.appendChild(item);
  });
}

function addEmptyLine(text) {
  const item = document.createElement("li");
  item.className = "empty";
  item.textContent = text;
  list.appendChild(item);
}

function setLoginStatus(message, type) {
  loginStatus.textContent = message;
  loginStatus.className = type;
}

```

style.css

```

/* Supabase guestbook styles (mobile-first) */
:root {
  --navy: #0B2E59;
  --blue: #0B5FBF;
  --cyan: #00B4D8;
  --grey: #5B6B7C;
  --light: #F5F9FD;
  --border: #D7E4F0;
}

* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: Arial, Helvetica, sans-serif;
  color: var(--navy);
  background: var(--light);
  line-height: 1.6;
}

.container { max-width: 640px; margin: 0 auto; padding: 40px 20px; }

.kicker { color: var(--cyan); font-weight: bold; letter-spacing: 2px; text-transform: uppercase; font-size: 0.8rem; }
h1 { font-size: 2rem; margin: 4px 0 8px; }
h2 { font-size: 1.3rem; margin: 32px 0 12px; }
h2 span { color: var(--grey); font-weight: normal; }
.intro { color: var(--grey); margin-bottom: 24px; }

.panel {
  display: flex;
  flex-direction: column;
  gap: 8px;
  background: #FFFFFF;
  border: 1px solid var(--border);
  border-radius: 14px;
  padding: 24px;
}

label { font-weight: bold; font-size: 0.95rem; }

input, textarea {
  padding: 11px 12px;
  border: 1px solid var(--border);
  border-radius: 8px;
  font-size: 1rem;
  font-family: inherit;
}
input:focus, textarea:focus { outline: 2px solid var(--cyan); border-color: var(--cyan); }

button {
  margin-top: 8px;
  padding: 12px;
  background: var(--blue);
  color: #FFFFFF;
  border: none;
  border-radius: 8px;
  font-size: 1rem;
  cursor: pointer;
}
button:hover { background: var(--navy); }
button:disabled { background: var(--grey); cursor: not-allowed; }

#status { min-height: 1.4em; font-weight: bold; }
#status.success { color: #1E8E5A; }
#status.error { color: #C0392B; }

```

```
#message-list { list-style: none; display: flex; flex-direction: column; gap: 12px; }

#message-list li {
  background: #FFFFFF;
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 14px 16px;
}
#message-list .time { color: var(--grey); font-size: 0.8rem; margin-left: 8px; }
#message-list p { margin-top: 4px; }
#message-list li.empty { color: var(--grey); font-style: italic; text-align: center; }
```

About the Instructor

Prof. Prakash Ukhalkar

Educator · Mentor · Machine Learning, AI & Innovation

Pimpri Chinchwad College of Engineering, Pune.

- GitHub: github.com/prakash-ukhalkar
- Website: prakash-ukhalkar.github.io
- LinkedIn: linkedin.com/in/prakash-ukhalkar